

Submission Date: 20th May 2026

Create a single zip file with all the files you want to submit. Name the file with the surname of all members of your group (e.g., `x_y_z.zip`). Upload your zip in the “Submission” folder of the corresponding week (Exercises → Week 4 - Multidimensional Deep Learning → Submission).

1 The Octree Trade-Off: Saving Memory vs. Access Speed

The **OctNet** structure presents a clever solution to the massive memory cost of dense 3D grids. It starts with a coarse grid and only subdivides cells that contain data, creating a tree-like structure. This saves memory but comes with a hidden cost in performance.

Conceptual: The Cost of Following Pointers

1. In a dense 3D array, `Grid[x][y][z]`, finding a voxel’s value is an immediate, one-step calculation. We call this an $O(1)$ operation. An Octree doesn’t allow for this. Describe the step-by-step process required to find the data for a specific coordinate (x, y, z) within an Octree.
2. Explain why this is much less efficient for a modern processor (like a GPU) compared to the direct memory access of a dense grid, even if the total number of voxels is far less.

Numerical: Calculating the Memory of an Octree

Let’s calculate the exact memory footprint for a specific tree. You start with a coarse grid of size $2 \times 2 \times 2$.

Memory Costs:

- An **Empty Leaf** (a cell with no data): **4 bytes**.
- A **Split Node** (a cell that is subdivided): **32 bytes** (to store 8 pointers to its children).
- A **Data Leaf** (a fine-grained voxel holding actual data): **4 bytes**.

Tree Structure:

1. At the **coarse level**, only cell **(0,0,0)** is a **Split Node**. The other 7 cells are Empty Leafs.
2. In the next level down, inside cell (0,0,0), only its sub-cell **(1,1,1)** is **also a Split Node**. The other 7 children are all Empty Leafs.
3. Finally, at the deepest level, the sub-cell (1,1,1) splits into 8 **Data Leafs**.

Calculate the total memory usage of this specific tree structure in Bytes.

2 The “Dilation Explosion” of Sparse Convolutions

In standard CNNs, we don’t worry about the number of active pixels increasing because the image is always a dense rectangle. In Sparse CNNs, however, the “Regular Sparse Convolution” (SC) acts like a dilation operator, thickening lines and filling holes. Let’s calculate exactly how fast a sparse signal loses its sparsity.

Conceptual:

Applying a $3 \times 3 \times 3$ sparse convolution (a cube-shaped kernel) dilates the input shape. For example:

1. If the input is a single point (one active voxel), what is the geometric shape of the output after one layer?
2. If the input is a thin line of voxels, what is the shape after one layer?
3. If the input is a flat plane of voxels, what is the shape after one layer?

Now, consider the growth from a single point. What is the side length of the active cube after L consecutive layers of $3 \times 3 \times 3$ convolutions?

Numerical:

Let's assume the input is a single active voxel. **After Layer 1** (Kernel $3 \times 3 \times 3$): The number of active voxels is $3^3 = 27$.

1. **After Layer 2** (Kernel $3 \times 3 \times 3$): Calculate the exact number of active voxels.
2. **After Layer 10**: Calculate the side length of the active cube and its total number of active voxels.
3. If your input grid was $64 \times 64 \times 64$, approximately how many layers would it take for this single point to "fill" the entire grid, effectively turning your sparse network into a dense one?

3 The "Disconnected Island" Problem and The "Strided Bridge" Solution

Submanifold Sparse Convolutions (SSC) are highly efficient as they preserve the number of active voxels. However, this efficiency comes at a cost: a strict topological limitation where information can only flow between existing, connected active sites. This can prevent a network from understanding an object as a single, coherent whole.

Part A: The Problem — Disconnected Islands

Conceptual: Imagine you are processing a scan of a table. Due to sensor noise or an obstruction, there is a **1-voxel gap** in the data between the table leg and the table top.

1. If you use a standard $3 \times 3 \times 3$ **Submanifold** Sparse Convolution, will the features from the leg ever influence the features of the table top, even after many layers?
2. Explain why or why not by describing how an SSC layer computes its output, focusing on the "active neighbor" rule.

Numerical Intuition: Let's model this in 1D. We have two disconnected segments:

- **"Leg":** Active voxels at indices $\{0, 1, 2\}$.
- **"Top":** Active voxels at indices $\{4, 5, 6\}$.
- **The Gap:** Voxel at index '3' is inactive.

Consider an SSC with a kernel of size 3, centered at $x = 2$ (the top of the leg). The kernel's receptive field covers input indices $\{1, 2, 3\}$.

1. Which of these input indices are active?
2. Does the output calculated at $x = 2$ receive any information from the "Top" segment?

Part B: The Solution — The Strided Bridge

Conceptual: To fix the "Disconnected Island" problem, we often use **Strided** Sparse Convolutions (or pooling). These operations are allowed to create new active sites at a coarser resolution, effectively bridging gaps in the data.

1. Explain intuitively how a strided convolution (e.g., with stride 2) acts as a "bridge." How does "zooming out" (reducing the resolution) help connect the leg and the top?

Numerical Intuition: Let's use the same disconnected data: active voxels at $\{0, 1, 2\}$ and $\{4, 5, 6\}$. We will now apply a **Strided** Sparse Convolution with a kernel of size 3 and a **stride of 2**.

The output coordinates will be in a new, coarser grid where $y = \lfloor x/2 \rfloor$.

1. A strided convolution can create an output if *any* voxel within its receptive field is active. The kernel centered at input $x = 2$ covers $\{1, 2, 3\}$. Is there an active voxel here? If so, at which output coordinate y is an output generated?

2. Now consider a kernel centered at input $x = 4$, covering $\{3, 4, 5\}$. Is there an active voxel here? If so, at which output coordinate y is an output generated?
3. By looking at your answers for the outputs at $y = 1$ and $y = 2$, are these two output voxels adjacent now? Has the gap been bridged in the new feature map?

4 Max Pooling

A core challenge with point clouds is their unordered nature. PointNet solves this by using a symmetric function—specifically, max pooling—to aggregate point features into a global signature. This design choice is powerful, but it also has a crucial consequence for how the network learns.

Part A: Why Max Pooling?

A simple summation is also a symmetric function, so why is max pooling generally preferred for achieving permutation invariance in shape classification?

1. **Conceptual:** Explain the key weakness of **Sum Pooling** when dealing with point clouds of varying densities (e.g., a scan with 1,000 points vs. one with 10,000 points of the same object). How does **Max Pooling** overcome this?
2. **Numerical:** An MLP maps a 3D point (x, y, z) to a 2D feature vector using the rule $h(x, y, z) = [x^2, z]$. You have two inputs:
 - **Input A (Dense):** Two points, $(2, 0, 1)$ and $(-2, 0, 1)$.
 - **Input B (Sparse):** One point, $(2, 0, 1)$.

Calculate the global feature vector for both inputs using first Sum Pooling, then Max Pooling. Which method produces a feature vector that is stable and robust to the change in point count?

Part B: How Does It Learn?

The choice of max pooling creates what is known as a "critical set," where the network focuses only on the most informative points that define a shape's boundary.

1. **Conceptual:** During backpropagation through a max pooling layer, what is the gradient for a point feature that was *not* the maximum in its channel? What does this imply about which points contribute to the network's learning in each training step?
2. **Numerical:** Your network processes three points, which are mapped to the single-channel features $F = [0.2, 0.9, 0.5]$. The global feature is $G = \max(F)$. The loss is $L = (G - 1)^2$. Calculate the gradient of the loss with respect to each of the three input features $(\partial L / \partial F_i)$. Which point(s) will have their weights updated?

5 Coding Exercise

In this week's coding exercise, you will implement some parts of PointNet for shape classification.

The provided instructions are for running the code in Google Colab.

- Download the exercise notebook *ADL_Ex02.ipynb* from Ilias.
- Upload the notebook *ADL_Ex02.ipynb* to Colab.
- Use the CPU runtime for coding and then switch later to the GPU for training to save GPU compute, which is limited in Colab. If you run out of GPU compute, use the CPU for training.
- Run the code cells step by step and follow the instructions in the notebook.
- Upload the filled-out notebook together with the outputs.

The detailed instructions for your tasks are in the notebook. Your tasks are:

1. Implement the transform to normalize the point cloud such that it is centered at the origin and all points lie within a unit sphere, i.e. the maximum distance of a point to the origin is 1.
2. Implement the transform to randomly rotate the whole point cloud around the z-axis (last axis). To do so, sample a random angle between 0° and 360° , create the rotation matrix and transform the point cloud with the rotation matrix. Hint: https://en.wikipedia.org/wiki/Rotation_matrix#Basic_3D_rotations
3. Implement the transform to add Gaussian noise with a standard deviation of 0.02 to each point separately.
4. Construct the layers of the T-Net proposed in PointNet and implement its forward pass.
5. Construct the layers of PointNet until the global feature extraction and implement the forward pass to get the global feature vector.
6. Train your PointNet implementation for 1 epoch. If your implementation is correct, the loss should reduce quickly and you should get an validation accuracy of around 50% – 70%.
7. Evaluate your trained PointNet by plotting a confusion matrix. Which classes are frequently predicted incorrectly? Which classes look similar for the network? Provide the answer in the notebook.